Topics	***Questions***	
Variables	1. Difference b/w var, let & const ? 2. What is javascript identifiers ? 3. General rule for constructing variable name ?	
Scope	1. What is scope and what are different scopes in JS ?	
JS Operators	1. Arithmetic Operators 2. Assignment Operators 3. Comparison Operators 4. String Operators 5. Logical Operators 6. Type Operators 7. Ternary Operators 8. Bitwise Operators(not important) 9. diff b/w == & ===	
Condition Statement	1. if / else 2. switch cases 3. ternary operator 4. optional chaining (?.) 5. nullish operator (??)	
Loops	1. while loop 2. do while loop 3. for loop 4. for ..in loop 5. for ..of loop 6. when we should use for ..in loop and for ..of loop ? 7. what is break and continue keyword and what are use of them ? 8. basic diff b/w while and do while loop ?	separate every digit from a number even / odd sum of all digits of number reverse digits of a number palindrome, prime, armstrong
ES 6	1. what are new features added in ES6 and further?	
Strict Mode	1. what is strict mode and how to use it in JS ? 2. why and when we should use strict mode ?	
Functions	1. what are functions in JS ? 2. different ways to declare functions ? 3. Diff functions in JS 4. what is IIFE ? 5. Diff b/w function declaration and function expression ? 6. reason why function expression are not hoisted ? 7. how many values a function can return ? 8. if we want multiple values from function then what we need to do ? 9. fiff b/w function parameter and function argument ? 10. Anonymous function	Write a function in JavaScript that takes an array of numbers and returns the sum of all positive numbers in the array Write a function in JavaScript that takes a string as input and returns a new string with all the vowels removed Write a function in JavaScript that takes an array of strings as input and returns a new array with the strings sorted in alphabetical order Write a function in JavaScript that checks if a string is a palindrome Write a function in JavaScript that finds the second highest number in an array of numbers Write a function in JavaScript that removes duplicates from an array Write a function in JavaScript that converts a given string to title case.

Topics	***Questions***	
Data types	1. String 2. Number 3. Bigint 4. Boolean 5. Undefined 6. Null 7. Symbol 8. Object 9. what is difference b/w primitive and non-primitive data types 10. difference b/w null and undefined ? 11. Typecasting or coercion ?	
String	1. different ways to create string ? 2. template string 3. why and when we use backslash in string (\ ) 4. strings are mutable or immutable ?	Reverse the string with and without method
String methods	1. length 2. at(position) 3. charAt(position) 4. charCodeAt(position) 5. diff b/w at() and charAt() ? 6. slice(start, end) 7. substring(start, end) 8. substr(start, length) 9. diff b/w substr, substring and slice ? 10. toUpperCase() 11. toLowerCase() 12. concat() 13. trim() 14. trimStart() 15. trimEnd() 16. repeat() 17. replace(x,y) 18. replaceAll(x,y) 19. split() 20. padStart() 21. padEnd() 22. indexOf() 23. lastIndexOf() 24. search() 24. diff b/w search and indexOf ? 25. match() 26. matchAll() 27. includes() 28. startsWith() 29. endsWith()	
Number	1. different behaviour of number when we perform arithmetic operation with numeric string 2. Nan 3. isNAN() 4. Infinity / -Infinity 5. toString() 6. why we should not start number with 0 while declaring number 7. Number Object 8. What are Numeric Separator in JS and how to use it ? 9. parseInt and parseFloat methods ?	

Topics	***Questions***	
Array	1. different ways to declare array 2. Array are Object and what typrof array is supported in JS ? 3. diff b/w array and object 4. what is spread operator and how to use it ? 5. what is destructuring and how to use it on a array ?	
Array Methods	1. length 2. toString 3. at 4. push 5. pop 6. shift 7. unshift 8. join 9. delete 10. concat 11. copyWithin 12. flat 13. slice 14. splice 15. toSpliced 16. indexOf 17. lastIndexOf 18. includes 19. find 20. findIndex 21. findLast 22. findLastIndex 23. sort 24. toSorted 25. reverse 26. toReversed 27. forEach 28. map 29. filter 30. reduce 31. reduceRight 32. flatMap 33. diff b/w map & forEach 34. every 35. some 36. from 37. keys 38. entries 39. with 40. What is spread operator and how to use it ? 41. Difference b/w spread operator and rest parameter ?	
Object	1. Objects are mutable or immutable ? 2. use of delete keyword ? 3. How to use spread operator in object ? 4. destructuring in object ? 5. diff b/w shallow copy and deep copy ?	

Topics	***Questions***	
Object Methods	1. Object.assign() 2. Object.create() 3. Object.defineProperties() 4. Object.defineProperty() 5. Object.entries() 6. Object.freeze() 7. Object.fromEntries() 8. Object.getOwnPropertyDescriptor() 9. Object.getOwnPropertyNames 10. Object.getPrototypeOf() 11. obj.hasOwnProperty() 12. Object.is 13. Object.isExtensible() 14. Object.isFrozen() 15. obj.keys() 16. Object.preventExtensions() 17. obj.propertyIsEnumerable() 18. Object.seal() 19. Object.isSealed() 20. Object.setPrototypeOf() 21. obj.toLocaleString() 22. obj.toString() 23. obj.valueOf() 24. Object.values()	
Events	1. event bubbling 2. event capturing 3. event propogation	
Errors	1. Different type of errors in JS ? (Reference, syntax, type, range, eval, URI (Uniform Resource Identifier) Error) 2. What is run time error in JS ? 3. difference between error and exception in JavaScript ? 4. what are try and catch statements and how to use it ? 5. what is finally statement ? 6. why we use throw statement and how to use it ? 7. difference between try catch and if-else statements ?	
Hoisting	1. what is hoisting ? 2. What is temporal dead zone ? 3. Does hoisting works in JS STRICT MODE ?	
Pure Function		
Higher Order function		
Recursion		
Prototype		
Modules	1.What is the difference between a module and a namespace ?	
JSON		
Closure		
Lexical Scope		
Curring		
Call, Apply & Bind		
Callback		

Topics	***Questions***	
Callback hell		
Promises	1. different states of a promise ? 2. Promise chaining 2. Promise.all()	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise
Asyn and await		
Window Object		
Debouncing		
Babel		
Event Loop		
Temporal dead zone		
WebPack		
this keyword		
arrow functions		
Debugging in JS		
Generator Function		
setTimeout and setInterval		
What is the event-driven programming paradigm? How does it work?		
Local Storage and session storage		
What are some common performance optimization techniques for JavaScript, and why would you want to use them?	1. Minification 2. Bundling 3. Caching 4. Lazy loading 5. Code optimization 6. Event delegation 7. Throttling and debouncing	
memoization		
Tree Shaking		

```
console.log([[0,1],[2,3]].reduce((acc,curr)=>{
   return acc.concat(curr);
},[1,2]))
```

```
let obj = {
    message:"hello",
    innermsg:function(){
        (function(){
            console.log(this.message)
        }())
    }
}

console.log(obj.innermsg())
```

```
function mul(x){
    return function(y){
        return function(z){
            return function(w){
                return function(p){
                    return x*y*z*w*p
                }
            }
        }
    }
}

console.log(mul(2)(3)(4)(5)(6))
```

```
let number = 10
let display = function(){
    console.log(number)
    var number = 20
}
display()
```

```
let i
for(i=0;i<3;i++){
    const log=()=>{
        console.log(i)
    }
    setTimeout(log,100)
}
```

```
let i
for(i=0;i<3;i++){
    const log=()=>{
        console.log(i)
    }
    setTimeout(log,100)
}
```

```
let i
for(i=0;i<3;i++){
    const log=()=>{
        console.log(i)
    }
    setTimeout(log,100)
}
```

```
function name(){
    if(1){
        var a=5
    }
    console.log(a)
}
name()
```

```
(function name(){
    var array = new Array("a",'b',"c",'d','e')
    array[10]='f'
    delete array[10]
    console.log(array.length)
}())
```

what are phases of event propagation

write ES6 feature of js

Array Addition Have the function ArrayAdostion(arr) take the array of numbers stored in and return the string true if any combination of numbers in the array (excluding the largest number) can be added up to equal the largest number in the array, otherwise return the string false.
For example: if arr contains (4, 6, 23, 10, 1, 3]
the output should return true
because 4 + 6 + 10 + 3 = 23.
The array will not be empty, will not contain all the same elements, and may contain negative numbers.
Examples • Input: (5,7,16, 1,21 Output: false

```
function findSubsetsThatSumToTarget(arr, target) {
  const result = [];


  function findSubsets(currentSubset, currentIndex, currentSum) {
      console.log({currentSubset, currentIndex, currentSum,target})
    // Base case: if the current sum equals the target, add the subset to the result
    if (currentSum === target) {
      result.push([...currentSubset]);
      return;
    }

    // If the current sum exceeds the target or we've processed all elements, backtrack
    if (currentSum > target || currentIndex === arr.length) {
        	console.log( currentSum> target || currentIndex === arr.length)

      return;
    }

    // Include the current element and recurse
    currentSubset.push(arr[currentIndex]);
    findSubsets(currentSubset, currentIndex + 1, currentSum + arr[currentIndex]);

    // Exclude the current element and recurse
    currentSubset.pop();
    findSubsets(currentSubset, currentIndex + 1, currentSum);
  }

  // Start the recursive process
  findSubsets([], 0, 0);
  return result;
}

// Example usage
const arr = [1, 4, 5, 6, 2, 3, 7];
const largestNumber = Math.max(...arr);
const subsets = findSubsetsThatSumToTarget(arr.filter(num => num !== largestNumber), largestNumber);

console.log(`Largest number: ${largestNumber}`);
console.log('Subsets that sum to the largest number:', subsets);
console.log('1');
console.log('2');
Promise.resolve(console.log('3'));
setTimeout(() => console.log('4'), 100);
console.log('5');
setTimeout(() => console.log('6'), 10);
Promise.resolve().then(() => console.log('7'));
setTimeout(() => console.log('8'), 0);
Promise.resolve(console.log('9'));
console.log('10');

// //1,2,3,5,9,10,7,8,6,4


// let arr=[4,1,5,2,6];
// for (let i=0;i<=arr.length-1;i++){
//     for(let j=i;j<=arr.length-i-1;j++){
//     if(arr[j]>arr[j+1]){
//         let temp=arr[j]
//         arr[j]=arr[j+1];

//           arr[j+1]=temp;
//     }

//     }
// };
// console.log(arr)
// arr.pop();
// console.log("Second Largest Number",arr[arr.length-1]);
// Test the function
const str1 = "army";
const str2 = "mary";

console.log(areCharactersSame(str1, str2)); // true
```

if all character in str1 and str2 are same it should return true otherwise false

```
function areCharactersSame(str1, str2) {
  return str2.split('').every(char => str1.includes(char));
}

// Test the function
const str1 = "army";
const str2 = "kary";

console.log(areCharactersSame(str1, str2)); // true

------------------------------------------------------------------------------------------------------------------------------------------
function areCharactersSame(str1, str2) {
  // If lengths are different, return false (as they can't have the same characters)
  if (str1.length !== str2.length) {
    return false;
  }

  // Create an object to store the character counts for str1
  const charCount = {};

  // Count characters in str1
  for (let char of str1) {
    charCount[char] = (charCount[char] || 0) + 1;
  }
  // Compare characters of str2 with str1
  for (let char of str2) {
    if (!charCount[char]) {
      // If a character from str2 is not in str1, return false
      return false;
    }
    charCount[char] -= 1; // Decrease the count for the matched character
  }
  // If all characters of str2 are found in str1 with correct counts, return true
  return true;
}

// Test the function
const str1 = "armyaq";
const str2 = "karyaq";

console.log(areCharactersSame(str1, str2)); // true
```

Variables

```
const testArray = [1, 2, 3];
testArray = [4, 5, 6];
console.log(testArray);
```

```
if(true){
    var a = 1
    let b =2
}
console.log(a)
console.log(b)
```

```
var a = 500
let b,c,d,e
e= 200

if(1){
    a=b=c=d=e
}

console.log(a,b,c,d,e)

if(0){
    a=b=c=d=e
}

console.log(a,b,c,d,e)
```

Scope

```
let x = 5;
{
  let x = 10;
  console.log(x);
}
console.log(x);
```

```
(function() {
  var a = b = 5;
})();

console.log(b);
```

```
function display() {
  var a = b = 10;
}
display();
console.log('b', typeof b === 'undefined');
console.log('a', typeof a === 'undefined');
```

```
function foo() {
  let a = b = 0;
  a++;
  return a;
}

foo();
console.log(typeof a)
console.log(typeof b)
```

```
var c = 10;

function foo(a){
    let b = 8;
    const d = 10;
    return function bar(){
        return a + d + c;
    }
}

const func = foo(7);
```

```
const fn = () => {
  a = 2;
  console.log(a);
}

fn();
```

```
for (var i = 0; i < 4; i++) {
  setTimeout(() => console.log(i), 0)
}


for(let i = 0; i < 5; i++){
    setTimeout(() => console.log(i), 1);
}
```

```
let count = 0;
(function immediate() {
  if (count === 0) {
    let count = 1;
    console.log(count);
  }
  console.log(count);
})();
```

```
function foo() {
  let a = b = 0;
  a++;
  return a;
}
foo();
console.log(typeof a)
console.log(typeof b)
```

JS Operators

```
let x = 1;

if (function f() {}) {
  x += typeof f;
}

console.log(x);
```

```
let x = [1, 2, 3];
let y = [1, 2, 3];
let z = y;

console.log(x == y);
console.log(x === y);
console.log(z == y);
console.log(z == x);
```

```
var x = 10;
var x = x++;
var x = ++x;
console.log(x);
```

```
var x = 10;
var x;
var x = ++x;
console.log(x);
```

```
var x = 10;
var y = x++;
var z = ++x;
console.log(x,y,z);
```

```
var x = 10;
var x = undefined;
var x = ++x;
console.log(x);
```

```
var x = 10;
var x = ++x;
var x = x++;
var y =x;
console.log(x);
console.log(y);
```

```
var x = 0;
var loop = function(){
while(x < 3){
console.log("I'm looping!");
x= x++;
}
};
loop();
```

```
var a = 50+20+"shubh"
var b = "shubh"+50+20

console.log(a,b)
```

```
var a = 'str'
var b = 10
var c = 20
var d = '30'

console.log(a+b+c)
console.log(a+b+c+d)
console.log(b+c+d)
```

```
var i = 5
i = i++ + ++i - --i - i--
console.log(i)
```

Loops

```
 let counter = 0;
for (var i = 1; i <= 10; i++) {
  counter+= i;
}
console.log(counter);
console.log(i);
```

```
const length = 4;
const numbers = [];
var i;
for (i = 0; i < length; i++) {
  // does nothing
}
{
  // a simple block
  numbers.push(i + 1);
}

numbers;
```

Functions

```
const obj = {
  a: "foo",
  b: function () {
    console.log(this.a);
  },
};

const c = obj.b;

obj.b();
c();
```

```
const x = { foo: 1 };
const y = { foo: 2 };

function addFoo(obj) {
  return obj.foo + 1;
}

console.log(addFoo(x));
 console.log(addFoo(y));
```

```
const number = 1;
const result = (function () {
  delete number;
  return number;
})();
console.log(result);
```

```
function arrayFromValue(item) {
  return
    [item];
}

console.log(arrayFromValue(10));
```

```
function setName() {
  this.name = 'devkode';
}
setName();
console.log(this.name);
```

```
function fun(num1) {
    var num2 = 6;
    function TDK() {
        var num3 = 10;
        console.log(num1 * num2 * num3)
    }
    return TDK;
}

var TeamDevKode = fun(5)
TeamDevKode ()
```

```
const animal = {
  animal_name: "cat",
  action: function () {
    console.log(`${this.animal_name} is doing action`);
  }
};

setTimeout(animal.action, 1000);
```

```
const animal = {
  animal_name: "cat",
  action: function () {
    console.log(`${this.animal_name} is doing action`);
  }
};

setTimeout(function () {
  animal.action();
}, 1000);
```

```
const animal = {
  animal_name: "cat",
  action: function () {
    console.log(`${this.animal_name} is doing action`);
  }
};

let func = animal.action.bind(animal);
setTimeout(func, 1000);
```

```
function getFunc() {
  let value = "Hey !";

  let func = new Function("console.log(value)");

  return func;
}

getFunc()();
```

```
function getFunc() {
  let value = "Hello Friends !";
  let func = () => {
    alert(value);
  };
  return func;
}

getFunc()();
```

```
x = 1;
function func() {
  this.x = 2;
  return x;
}
let a = new func();
console.log(a.x);
```

```
let arr = Array.from(Array(10).keys());
function func(a) {
  console.log(arguments.length);
}
func(arr);
func(...arr);
```

```
function func(a, b) {
  arguments[1] = 2;
  console.log(b);
}
func(1);
```

```
var x = 3;
var obj = {
  x: 2,
  foo: {
    x: 1,
    bar: function () {
      return this.x;
    },
  },
};
var func = obj.foo.bar;
console.log(func());
console.log(obj.foo.bar());
```

```
var foo = function foo() {
  console.log(foo === foo);
};
foo();
```

```
function getAge() {
  'use strict';
  age = 21;
  console.log(age);
}

getAge();
```

```
const obj = { 1: 'a', 2: 'b', 3: 'c' };
const set = new Set([1, 2, 3, 4, 5]);

obj.hasOwnProperty('1');
obj.hasOwnProperty(1);
set.has('1');
set.has(1);
```

```
var num = "10";
(function () {
        console.log(num);
  var num = "50";
  console.log(num);}
  })();
```

Data types

```
console.log(null===undefined)
console.log(null==undefined)
console.log(2 + true);
console.log([] == ![]);
console.log('5' - - '3');
console.log(   2 + "2" );
console.log( "2" + "2" );

console.log(   2 - "2" );
console.log( "2" - "2" );
console.log( "A" - "A" );
```

```
console.log( 10 < 20 < 30 );
console.log( 30 > 20 > 10 );
console.log( null == 0 );
console.log( null > 0  );
console.log( null >= 0 );
console.log(2+undefined) //
console.log(2+null)
console.log(2+NaN+"2")
```

String

```
var name = new String("shubham")

console.log(name)
console.log(typeof name)
```

String methods

Number

```
console.log(isNaN(1+null))
console.log(isNaN(1+undefined))
console.log(NaN === NaN);
```

```
let value = NaN;
if(value) {
    console.log( "Truthy value");
} else {
    console.log( "Falsy value" );
}
```

Array

```
const arr = [1, 2, 3, 4, 5];

for (var i = 0; i < arr.length; i++) {
  setTimeout(function () {
    console.log(i);
  }, 1000);
}
```

```
const a = [1, 2, 3];
const b = a;

b.push(4);

console.log(a);
console.log(b);
```

```
const arr1 = [1, 2, 3];
const arr2 = [...arr1];

arr2.push(4);

console.log(arr1);
console.log(arr2);
```

```
const arr = [1, 2, 3];
const [a, b, c] = arr;

console.log(a);
console.log(b);
console.log(c);
```

```
const arr = [1, 2, 3, 4, 5];

for (let i = 0; i < arr.length; i++) {
  setTimeout(function () {
    console.log(i);
  }, 1000);
}
```

```
let array = [1, 2, 3, 4, 5, 6];
let [a, b, , ...rest] = array;

console.log(a)
console.log(b)
console.log(c)
console.log(rest)
let c;
```

```
let array1 = [1, 2, 3, 4, 5, 6];
let array2 = array1
array2.push(7, 8, 9, 10)
console.log(array1)
console.log(array2.sort())
```

```
let array = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
function checkNum(num) {
    return num >= 4;
}

let newArray1 = [array.map((item) => {
    return item
})].filter(checkNum)

let newArray2 = array.filter(checkNum).map((item) => {
    return item
})

console.log(newArray1)
console.log(newArray2)
```

```
let array = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
function checkNum(num) {
    return num > 4;
}
let newArray = [array.filter(checkNum).map((item, index) => {
    return item - array[index]
}).reduce((acc, curr) => {
    return acc + curr
})]

console.log(newArray)
```

```
const doSomething = (a, b) => {
    return [a + b, a - b, a * b, a / b]
}
let [...rest] = doSomething(4, 2)

console.log(rest * rest)
```

```
const doSomething = (a, b) => {
    return [a + b, a - b, a * b, a / b]
}
let someVariable = 5;
let [...rest] = doSomething(4, 2)

someVariable = rest.reduce((acc, total) => {
  return 0 + total;
});

console.log(someVariable)
```

```
const doSomething = (a, b) => {
    return [a + b, a - b, a * b, a / b]
}
let someVariable = 5;
let [...rest] = doSomething(4, 2)

someVariable = rest.reduce((acc = 1, total) => {
  return acc + total;
});

console.log(someVariable)
```

```
const nums = [1, 2, 3, 4, 5, 6];

const res = nums.forEach((item) => {
  return item * item;
});

console.log(res);
```

```
let array = [1, 2, 3];
let array2 = array;
let array3 = [...array];

console.log(array.concat(4));
console.log(array2.concat(5));
console.log(array3.concat(6, 7));
```

```
const clothes = ['jacket', 't-shirt'];
clothes.length = 0;

console.log(clothes[0]);
```

Array Methods

```
let a = { x: 1 };
let b = { x: 2 };
let c = { x: 3 };
let d = { x: 4 };
let e = { x: 5 };
let arr = [a, b, c, d, e];

arr.forEach((obj) => (obj.x = obj.x * 2));

console.log(a.x, b.x, c.x, d.x, e.x);
```

```
const testArray = [1, 2, 3, 4, 5];
const res = testArray.reduce((acc, curr) => {
  if (curr % 2 === 0) {
      return acc + curr;
  }
  return acc;
}, 0);
console.log(res);
```

```
const array = [10, 20, 30, 40];
const result = array.map((num) => num / 2).filter((num) => num >= 15);
console.log(result);
```

```
const companies = [
  { id: "1", name: "Facebook" },
  { id: "2", name: "Apple" },
  { id: "3", name: "Google" },
];

companies.sort((a, b) => (a.name > b.name ? -1 : 1));
console.log(companies);
```

```
const clothes = ['jacket', 't-shirt'];
clothes.length = 0;

clothes[0];
```

```
const elements = [1, 2, 3, 4, 5];

elements.forEach(element => {
  console.log(element);

  if(element == 2){
    return;
    //break;
    //continue;
  }
})
```

Object

```
const obj = { a: 1, b: 2, c: 3 };
const { a, b } = obj;
console.log(a, b);
```

```
const obj = {
  name: "Conner",
  age: 27,
  greet: () => {
       console.log(`Hey, my name is ${this.name}`);
  },
};
obj.greet();
```

```
const obj = {
  a: 1,
  b: 2,
  c: {
    a: 3,
    b: 4,
  },
};

const {
  a,
  b,
  c: { a: nestedA },
} = obj;

console.log(a, b, nestedA);
```

```
let obj = { name: "John", age: 25 };
let newObj = { ...obj, age: 30 };

console.log(obj.age);
console.log(newObj.age);
```

```
let a = {
  x: 1,
  y: 2,
};
let b = a;
a.x = 5;
console.log(a);
console.log(b);
```

```
let obj1 = { key: "value" };
let obj2 = obj1;
let obj3 = obj2;

obj1.key = "new value";
obj2 = { key: "another value" };

console.log(obj1.key, obj2.key, obj3.key);
```

```
let a = {};
let b = { key: "b" };
let c = { key: "c" };

a[b] = 123;
a[c] = 456;

console.log(a[b]);
```

```
const name = "John";
const age = 25;

const user = { name, age };
console.log(user);
```

```
const user = {
  name: 'Raj',
  location: {
    city: 'NY',
    state: 'NY'
  }
};
const copy = Object.assign({}, user);
// OR
// const copy = { ...user };
copy.location.city = 'Albany';
console.log('original: ', user.location);
console.log('copy:', copy.location);
```

```
const object = {
  message: 'Hello, World!',

  getMessage() {
    const message = 'Hello, Earth!';
    return this.message;
  }
};

console.log(object.getMessage());
```

```
let a=3; let b = new Number(3); let c=3;

console.log(a == b); console.log(a === b); console.log(b === c);
```

```
const telegramGroup = {
   name: 'TeamDevkode'
}
const { name: TDK } = telegramGroup;

console.log(TDK);
```

```
let myName = "Sunny";
let groupName = myName;

myName = "DevKode";
console.log(groupName);


const obj1 = {
    id:1,
    name:"Sunny",
}

const obj2 = obj1;
obj2.name = 'DevKode';
console.log(obj1);
```

```
 function greet(person) {
  if (person == { name: 'Narendra Modi' }) {
    return 'hey Narendra Modi'
  } else {
    return 'hey Donald Trump'
  }
}

console.log(greet({ name: 'Narendra Modi' }))
```

```
const sample = ["xyz", "abc", "test", "ryan", "apple"];
delete sample[3];
console.log(sample.length);
```

```
const fruits_obj = { 0 : "apple" , 1 : "mango" ,  2:"banana"};
const fruits_arr = ["apple","mango", "banana"];

console.log(typeof fruits_obj);
console.log(typeof fruits_arr);
console.log(fruits_obj == fruits_arr);
```

Object Methods

```
const person = {
  name: ""John"",
  age: 30,
};

Object.freeze(person);
person.age = 40;

console.log(person.age);
```

Events

https://github.co

Hoisting

```
let num = 0;

function test() {
  var num = 1;
  return num;
}

console.log(test());
console.log(num);
```

```
let x = 10;

function outer() {
  console.log(x);

  if (false) {
    var x = 20;
  }
}

outer();
```

```
let x = 10;

function updateX() {
  if (true) {
    let x = 20;
    console.log(x);
  }
  console.log(x);
}

updateX();
```

```
function calculateSum() {
  console.log(result);
  var num1 = 5;
  let num2 = 10;
  const num3 = 15;
  var result = num1 + num2 + num3;
}

calculateSum();
```

```
const x = 10;

function foo() {
  console.log(x);
  const x = 20;
}

foo();
```

```
var name = "Shubham";
var name;

console.log(name);
```

```
num1 = 100;
console.log(num1)
var num1
console.log(num2)
var num2 = 200
```

```
var num = 500
const result = function (){
    console.log(num)
    var num = 1000
}

result()
```

```
console.log(typeof foo);

function foo() {
  return "bar";
}
var foo = "bar";


function foo() {
  return "bar";
}
var foo;

console.log(typeof foo);
```

```
if (true) {
  function foo() {
    console.log(1);
  }
} else {
  function foo() {
    console.log(2);
  }
}

foo();
```

```
function foo() {
  bar();

  return;

  function bar() {
    console.log("bar");
  }
}

foo();
```

```
function foo(x) {
  x();

  function x() {
    console.log("foo");
  }
}

foo(function() {
  console.log("bar")
});
```

```
foo();

function foo() {
  console.log(1);
}

var foo = function() {
  console.log(2);
};

function foo() {
  console.log(3);
}

foo();
```

```
function animal(){
  console.log("Cat");
}

var otherAnimal;

animal();
otherAnimal();

otherAnimal = function() {
  console.log("Dog");
}
```

```
b = function a(){};
var a = b = 6;
a = function b(){};
function b() {};
function a() {};
console.log(a,b);
```

```
var a = 10;
console.log("line number 2", a);

function fn() {
  console.log("line number 4", a);
  var a = 20;
  a++;
  console.log("line number 7", a);
  if (a) {
    var a = 30;
    a++;
    console.log("line number 11", a);
  }
  console.log("line number 13", a);
}
fn();
console.log("line number 2", a);
```

```
function foo() {
  let a = b = 0;
  a++;
  return a;
}

foo();
console.log(typeof a);
console.log(typeof b);
```

Errors

Pure Function

Higher Order function

Recursion

Prototype

Modules

JSON

Closure

```
function foo() {
  let x = 1;

  function bar() {
    let y = 2;
    console.log(x + y);
  }

  bar();
}

foo();
```

Curring

Call, Apply & Bind

Callback

```
const add = (a = 1, b = 2) => a + b;
console.log(add());
console.log(add(5));
console.log(add(undefined, 10));
```

Callback hell

Promises

```
const promise1 = Promise.resolve("One");
const promise2 = new Promise((resolve) => setTimeout(() => resolve("Two"), 1000));
const promise3 = Promise.reject("Three");
 Promise.allSettled([promise1, promise2, promise3]).then((results) => console.log(results));
```

```
console.log('Start');

Promise.resolve().then(() => console.log('Promise 1'));
setTimeout(() => console.log('Timeout 1'), 0);

Promise.resolve().then(() => console.log('Promise 2'));
setTimeout(() => console.log('Timeout 2'), 0);

console.log('End');
```

```
const validateUsernamePassword = (username, password) => {
        return new Promise((resolve, reject) => {
                if (username == "ABC" && password == "123") {
                        resolve("SUCCESS!");
                } else {
                        reject("FAILURE!");
                }
        });
};

validateUsernamePassword("ABC", "123")
        .then((msg) => {
                console.log(msg);
        })
        .catch((err) => {
                console.log(err);
        });
```

```
let setTimeoutResolveReject = new Promise((resolve, reject) => {
        setTimeout(() => {
                resolve("YES");
        }, 3000);
        setTimeout(() => {
                reject("NO")
        }, 2000);
});

setTimeoutResolveReject
.then((msg) => {
        console.log(msg);
})
.catch((err) => {
        console.log(err);
})
A: YES
B: NO
C: NO\nYES
D: YES\nNO
```

```
let examplePromise = new Promise((resolve, reject) => {
        let timeout = 3000;
        let randomWaitTime = Math.floor(Math.random() * 5000) + 1;
        console.log(randomWaitTime);
        if (randomWaitTime <= timeout) {
                setTimeout(() => {
                        resolve();
                }, randomWaitTime);
        } else {
                setTimeout(() => {
                        reject();
                }, randomWaitTime);
        }
});

examplePromise
        .then(() => {
                console.log("RESOLVED!");
        })
        .catch(() => {
                console.log("REJECTED!");
        });
```

```
new Promise((resolve, reject) => {
  console.log(4)
  resolve(5)
  console.log(6)
}).then(() => console.log(7))
.catch(() => console.log(8))
.then(() => console.log(9))
.catch(() => console.log(10))
.then(() => console.log(11))
.then(console.log)
.finally(() => console.log(12))
```

```
const promise_1 = Promise.resolve('First');
const promise_2 = Promise.resolve('Second');
const promise_3 = Promise.reject('Third');
const promise_4 = Promise.resolve('Fourth');

const runPromises = async () => {
        const res1  = await Promise.all([promise_1, promise_2])
        const res2  = await Promise.all([promise_3, promise_4])
        return [res1, res2];
}

runPromises()
        .then(res => console.log(res))
        .catch(err => console.log(err))
```

```
var p = new Promise((resolve, reject) => {
  reject(Error('The Fails!'))
})
p.catch(error => console.log(error.message))
p.catch(error => console.log(error.message))
```

```
console.log('initial');
setTimeout(function() {
   console.log('setTimeout');
}, 0);
var promise = new Promise(function(resolve, reject) {
   resolve();
});
promise.then(function(resolve) {
   console.log('1st Promise');
})
.then(function(resolve) {
   console.log('2nd Promise');
});
console.log('final');
```

```
let promise = new Promise( (resolve)=>{
console. log('Step 1');
resolve('Step 2');
} )
promise. then ( (data)=>{
console. log(data);
console. log ('Step 3')
```

```
let promise = new Promise (resolve,reject)=
console. log( 'Step 1'); reject (new Error( 'Step 2'));
})
promise. then ( (data)=>{
console. log(data);
} )
console. log('Step 3')
```

https://github.com/devkodeio/javascript-interview-questions/blob/main/code-snippets/promises.md

Asyn and await

```
async function asyncFunc() {
   console.log('A');
   await new Promise(resolve => setTimeout(resolve, 1000));
   console.log('B');
}
console.log('C');
asyncFunc();
console.log('D');
```

Window Object

Debouncing

Babel

Event Loop

```
console.log(1)

setTimeout(()=>{console.log(2)},100)
let myPromise = new Promise(function(myResolve, myReject) {
myResolve()
});

myPromise.then(
  function(value) {console.log(3)}
);
console.log(4)
```

Temporal dead zone

WebPack

this keyword

```
const object1 = {
  a: 10,
  b: 20,
  c: function () {
        console.log(this.a + this.b);
  },
};
const func = object1.c;
func();
```

```
 const person = {
  firstName: "Shubham",
  lastName: "Sharma",
  getFullName: function () {
        return this.firstName + " " + this.lastName;
  },
};
console.log(person.getFullName());
```

```
"const object = {
  message: 'Hello, World!',

  getMessage() {
    const message = 'Hello, Earth!';
    return this.message;
  }
};

console.log(object.getMessage());"
```

arrow functions

setTimeout and setInterval

```
let x = true
setTimeout (()=>{
x= false
})
while (x)
{
console.log("hello")
}
```

```
let x = true
let count =0;
setTimeout(()=>{
x= false;
},2000)
setInterval(()=>{
if(x)
{
console.log(count++)
}
},200)
```

```
const arr = [10, 12, 15, 21];
for (var i = 0; i < arr.length; i++) {
  setTimeout(function() {
    console.log('Index: ' + i + ', element: ' + arr[i]);
  }, 3000);
}
```

memoization

[[1,0,1],[1,1,0],[0,0,1]] print it column wise

```
let arr1 = [1, 2, 3];
let arr2 = [4, 5, 6];  // concat it using spread operator
```

```
const names = ["pawan", "sashikant", "Sourabh", "harish"]; // find logest string in array of strings
```

```
const passengersData = [
    {
      id: 1,
      passengerName: "Freddie Mercury",
      isVegetarianOrVegan: false,
      connectedFlights: 2,
    },
    {
      id: 2,
      passengerName: "Amy Winehouse",
      isVegetarianOrVegan: true,
      connectedFlights: 4,
    },
    {
      id: 3,
      passengerName: "Kurt Cobain",
      isVegetarianOrVegan: true,
      connectedFlights: 3,
    },
    {
      id: 3,
      passengerName: "Michael Jackson",
      isVegetarianOrVegan: true,
      connectedFlights: 1,
    },
   ];
```

How to Get Passenger's Names - using map method,using for..of loop,using for loop
How to Get Vegetarian/Vegan Singers - using for..of loop, using filter and map together
How to Sort Passengers by Connecting Flights

Write a JavaScript function to get the first element of an array. Passing a parameter 'n' will return the first 'n' elements of the array. Go to the editor

Test Data :

```
console.log(first([7, 9, 0, -2]));
console.log(first([],3));
console.log(first([7, 9, 0, -2],3));
console.log(first([7, 9, 0, -2],6));
console.log(first([7, 9, 0, -2],-3));
```

Expected Output :

```
7
[]
[7, 9, 0]
[7, 9, 0, -2]
[]
```

Write a JavaScript function to get the last element of an array. Passing a parameter 'n' will return the last 'n' elements of the array. Go to the editor

```
Test Data :
console.log(last([7, 9, 0, -2]));
console.log(last([7, 9, 0, -2],3));
console.log(last([7, 9, 0, -2],6));
Expected Output :
-2
[9, 0, -2]
[7, 9, 0, -2]
```

Write a simple JavaScript program to join all elements of the following array into a string. Go to the editor

```
Sample array : myColor = ["Red", "Green", "White", "Black"];
Expected Output :
"Red,Green,White,Black"
"Red,Green,White,Black"
"Red+Green+White+Black"
```

Write a JavaScript program which accept a number as input and insert dashes (-) between each two even numbers. For example if you accept 025468 the output should be 0-254-6-8

Write a JavaScript program to sort the items of an array. Go to the editor

```
Sample array : var arr1 = [ -3, 8, 7, 6, 5, -4, 3, 2, 1 ];
Sample Output : -4,-3,1,2,3,5,6,7,8
```

with and without sort method

Write a JavaScript program to find the most frequent item of an array. Go to the editor

```
Sample array : var arr1=[3, 'a', 'a', 'a', 2, 3, 'a', 3, 'a', 2, 4, 9, 3];
Sample Output : a ( 5 times )
```

Write a JavaScript program which accept a string as input and swap the case of each character. For example if you input 'The Quick Brown Fox' the output should be 'tHE qUICK bROWN fOX'.

Swapping Numbers without Variable
Swapping with extra variable
Swapping using destructuring

Find num is Positive or Negative

Even or Odd

Nested if/else validate userId & pwd

Calci using if/else statements

Get day using switch statement

iterate array using diff methods

iterate string using diff methods

Separate Every digit of a number

Even or odd every digit of a number

Sum of every Digit of a Number

Reverse the digits of a number - using loop, using methods of array

Palindrome number or not

Prime number or not

Armstong number or not

Factorial of number

Factorial of all number b/w 1-10

Take Size & elements of arr from user & find biggest element, Second biggest using for and for ..of loop

Check each element of array is Even or Odd and sum of all even/odd

```
      var arr = [12, 35, 1, 10, 28, 7];
```

Sum of Prime and Non-Prime

```
      var arr = [12, 35, 1, 10, 28, 7];
```

Find Repeated element with their count

Print reverse of array without method and with method

Compare 2 arrays are equal or not ==>

```
      var arr1 = [22, 34, 45, 67];
      var arr2 = [22, 34, 45, 67];
```

Separate string,number,float values from array

find factorial of each element in array

How to recognise Array as typeOf operator show Object for array

Biggest Element & Lowest Element - using sort() , without sort method

Biggest, 2nd Bigggest, Lowest, 2nd Lowest - using sort() , without sort()

reverse a string using recursion

capitalize first and last letter of each word in string

logest word in a str

reverse each letter of each word of string

How to find object key in JavaScript

```
//findMissingNumber([1, 3, 5, 6, 7, 8, 9, 10, 2, 4, 12])
const findMissingNumber=(args)=>{
    let len=args.length;
    let sortArr=args.sort((a,b)=>a-b);
    let maxNo=sortArr[sortArr.length-1];
    let minNo=sortArr[0];
    for(let i=minNo;i<maxNo;i++){
        if(!sortArr.includes(i+1)){
          return i+1;
        }

    }
}
let missing=findMissingNumber([12,15,16,11,14,19,20,17,18]);
console.log(missing)
```

https://github.com/sudheerj/reactjs-interview-questions?tab=readme-ov-file#what-is-react	repo for react interview questions
Questions	**Answers**
1. What is React?	Open-source JavaScript library, Developers to create reusable UI components, Virtual DOM, Unidirectional Data Flow, State and Props, JSX (JavaScript XML)
2. What is useMemo ?	React Hook that helps optimize performance by memoizing (saving) the result of a computation, Sorting, filtering, or large computations To avoid recalculations for unchanged data
3. What is JSX ?	JavaScript XML, Allows you to write HTML-like code within JavaScript, JSX is not valid JavaScript. React uses a compiler (like Babel) to transform JSX into regular JavaScript
4. What is DOM ?	Document Object Model is a programming interface that represents the structure of a web document (like an HTML or XML file) as a tree of objects, **DOM METHOD:** document.getElementById(), document.querySelector(), document.createElement(), element.appendChild(), element.removeChild(), **DOM TYPE:** HTML DOM, CSS DOM, SVG DOM
5. What is Virtual Dom ?	Copy of the real webpage (DOM) that React uses to work faster and smarter, Works faster, Updates only the necessary parts of the page, Keeps the app smooth and efficient
6. What is component life cycle of React class component ?	The component lifecycle in React refers to the series of stages a React class component goes through from creation to removal. **Mounting** (Component is being created and added to the DOM) **Updating** (Component is being updated due to state or props changes) **Unmounting** (Component is being removed from the DOM)
7. What are fragments in react ?	Allow you to group multiple elements without adding extra nodes to the DOM Fragments do not appear in the DOM - No Additional DOM Nodes Lightweight, improve performance, and keep your code cleaner
8. What are props in react ?	Data is passed from a parent component to a child component Read-only and allow components to be dynamic and reusable
9. What are synthetic events in react ?	Handle events (like clicks, form submissions, etc.) in a consistent way across different web browsers **Common SyntheticEvent Methods:** **preventDefault():** Prevents the default action (e.g., stopping a form from submitting). **stopPropagation():** Stops the event from bubbling up to parent elements. **persist():** Prevents the event object from being recycled by React, allowing you to access it later.
10. What are the difference between Package.json and Package.lock.json ?	**Package.json:** Lists project info and dependencies, Manually created or generated using npm init **Package.lock.json:** Locks exact versions of dependencies or Describe dependencies, Automatically created/updated by npm when installing packages.

11. What are the differences between client side and server side rendering ?	**Client Side:** Rendering happens in the browser after the JavaScript loads. Less SEO-friendly eg: A React app built with Create React App **Server Side:** Rendering happens on the server, and HTML is sent to the browser. SEO-friendly eg: A Next.js app using getServerSideProps
12. What is state in Reactjs?	State is an object that holds dynamic data about a component Allows the component to re-render whenever the data changes
13. What are the differences between State and Props in react ?	**State** is used to store data that can change within a component, and changes to state trigger a re-render of the component. **Props** are used to pass data from parent to child components, and they cannot be modified by the child component.
14. What is props drilling ?	Process of passing data (via props) from a parent component to a deeply nested child component in React
15. What are the disadvantages of props drilling and How we can avoid props drilling ?	**Disadvantage:** Code Complexity Difficult to Manage Increased Risk of Errors Unnecessary Re-Renders **Solutions to Avoid Props Drilling:** **Context API** allows you to share state or data across multiple components without passing props through every level of the component tree. **state management,** tools like Redux or Recoil can help manage global state that can be accessed by any component, eliminating the need for props drilling.
16. What are Pure components in React ?	Component that only re-renders when its props or state have changed. Pure components help improve performance by preventing unnecessary renders.
17. What are Ref's in React?	**refs (short for references)** are used to access and interact with DOM elements or React components directly. They are created using React.createRef() (for class components) or useRef() (for function components).
18. What is meant by forward ref ?	Normally, you can't directly control a child component. With Forward Ref, the parent can focus the input inside the child without extra work Use it when you need direct control (e.g., focus, animations) over a child component's element.
19. What are Error boundaries ?	Error boundaries catch errors only during rendering, in lifecycle methods, and in constructor of their child components. They do not catch errors inside event handlers, asynchronous code, or server-side rendering.
20. What are Higher order components in react ?	HOC is a function that takes a component and returns a new component with additional features. Use HOCs for authentication, data fetching, or enhancing styles.
21. What are the differences between controlled and uncontrolled components ?	**Controlled:** The component uses React state to track and control the form's input value Data is accessed via React's state **UnControlled**: The input's value is managed by the DOM, not React. Data is accessed using refs to get values directly from the DOM
22. What is useCallback ?	React recreates functions every time a component renders. useCallback keeps a cached version of the function until its dependencies change. This reduces unnecessary re-renders and improves performance.
23. What are the differences between useMemo and useCallback ?	**useMemo:** Memoizes a computed value, Returns the result of the memoized computation. **useC**allback: Memoizes a function Returns the memoized function itself.
24. What are keys in React ?	Keys are special attributes used to identify elements in a list, React uses the key to keep track of each item
25. What is Lazy loading in React ?	Improve application performance by loading components or resources only when they are needed React provides the built-in React.lazy function for lazy loading components.

26. What is suspense in React ?	Allows you to pause rendering of part of your component tree until some asynchronous operation (like data fetching or lazy loading) is complete Handling Lazy Loading Suspense is like a waiting room in your app where you can tell React to pause until certain content is ready (like a component or data). While waiting, React shows a loading spinner or message, so the user knows the app is still working on it.
27. What are custom hooks ?	Custom hooks are user-defined functions that allow you to reuse logic across multiple components
28. What is useReducer hook ?	React hook that is used to manage state in a more complex way than useState **useReducer** is like a manager for your state. If you have multiple things to track in your app (like a counter, user input, or form data), it keeps everything organized and makes sure the state is updated in the right way
29. What are Portals in react ?	Portals provide a way to render children outside their parent component's DOM Portals are useful for rendering things like modals, tooltips, dropdowns, or anything else that needs to break free from the typical parent-child relationship in the DOM
30. What is context in react ?	Context is like a global container where you can store data and share it across your app without passing it down through every component
32. What is the purpose of callback function as an argument of setState()?	**Callback function** passed as an argument to setState in React serves a critical purpose: it executes code after the state update has been completed and the component has re-rendered React’s setState is asynchronous. This means that state updates may not be immediately reflected after calling setState, as React batches updates for performance reasons. The callback ensures you can execute code after the state has been applied and the DOM has been updated
34. Which lifecycle hooks in class component are replaced with useEffect in functional components ?	The useEffect hook in functional components replaces these lifecycle methods from class components: componentDidMount: Use useEffect(() => { ... }, []); Runs once after the component mounts. componentDidUpdate: Use useEffect(() => { ... }, [dependencies]); Runs after updates when specified dependencies change. componentWillUnmount: Use useEffect(() => { return () => { ... }; }, []); Handles cleanup before the component unmounts. Combined Behaviors: Mount + Update: useEffect(() => { ... }); Mount + Unmount: useEffect(() => { return () => { ... }; }, []); Other Lifecycle Methods: shouldComponentUpdate: Use React.memo. getSnapshotBeforeUpdate: Use useLayoutEffect.
35. What is Strict mode in react ?	Strict Mode is a tool in React that helps developers identify potential issues in their applications **When to Use Strict Mode** During development to catch potential bugs early. To ensure your codebase is compatible with future React updates.

36. What are the different ways to pass data from child component to parent component in react ?	**Callback Functions** - Simple parent-child communication. Most common and recommended for direct use.
	State Management Libraries - For complex apps with deeply nested components sharing state.
	Refs - When the parent needs direct access to child's data or methods.
	Events - For decoupled components or global event handling.
	Context API - For a shared state between parent and child with React's built-in tools.
	
	Ways to Pass Data from Child to Parent in React
	
	Callback Functions (Most Common)
	Parent passes a function as a prop; child calls it with the data.
	<Child sendData={(data) => console.log(data)} />
	
	State Management Libraries (e.g., Redux, Context)
	Share state between parent and child using global or contextual state.
	const { setData } = React.useContext(DataContext);
	
	Refs
	Parent passes a ref to directly access child methods or data.
	const childRef = useRef(); childRef.current.getData();
	
	Custom Events
	Use events like EventEmitter for decoupled communication.
	
	Context API
	Use shared context between parent and child for state updates.

37. How to send data from child to parent using callback functions ?	You can send data from a child component to its parent by passing a callback function from the parent to the child as a prop. The child then calls this function and passes the data as an argument. **Steps:** Create a callback function in the parent component to handle the data from the child. Pass the callback function as a prop to the child component. Invoke the callback function in the child, passing the data to it.
38. How to send the data from child component to parent using useRef ?	You can use useRef to allow the parent component to directly access methods or data in the child component. The parent can then "pull" data from the child when needed. **Steps:** Create a ref in the parent using useRef. Pass the ref to the child using React.forwardRef. Expose methods or data in the child using useImperativeHandle. Access the child's data in the parent via the ref.
39. How do you optimize your react application ?	**Render Minimization:** Use React.memo, useMemo, and useCallback. **Code Splitting:** Lazy-load components with React.lazy and Suspense. **List Optimization:** Use virtualization for large datasets. **State Management:** Keep state local and use efficient tools like Redux Toolkit.
40. How would you consume a RESTful JSON API in reactjs?	***Key Points:*** **useEffect:** Fetches data on component mount. **State**: Manage loading, data, and error. **Axios or fetch**: Use whichever works best for your needs.

<table>
<tr><th></th><th></th></tr>
<tr><td>43.different design patterns used in react?</td><td>**Container/Presentational:** Separate logic and rendering.
HOC: Wraps a component to add extra behavior.
Render Props: Shares logic via a function passed as a prop.
Lift State Up: Share state between components by lifting it to the common ancestor.
Provider (Context API): Share state globally without prop drilling.
Module: Organize code into smaller, reusable pieces.
Observer: Components listen to state changes or events.
Compound Components: Use parent-child relationship for complex UIs.</td></tr>
<tr><td>44. context api vs redux</td><td>**Use Context API** for simple state management or small to medium-sized applications.
Use Redux for complex, large-scale apps where state management needs to be robust, scalable, and predictable.

<table>
<tr><th>*Feature*</th><th>*Context API*</th><th>*Redux*</th></tr>
<tr><td>Use Case</td><td>Small to medium apps, global settings</td><td>Large, complex apps with complex state</td></tr>
<tr><td>Performance</td><td>Can cause unnecessary re-renders</td><td>Optimized with selective re-renders</td></tr>
<tr><td>Setup Complexity</td><td>Simple setup, minimal boilerplate</td><td>More setup required, complex</td></tr>
<tr><td>Scalability</td><td>Not ideal for large apps</td><td>Highly scalable for large apps</td></tr>
<tr><td>Tooling/Debugging</td><td>Basic, limited tooling</td><td>Advanced tooling with DevTools</td></tr>
<tr><td>Learning Curve</td><td>Low</td><td>Steeper due to actions, reducers, etc.</td></tr>
<tr><td>Async Actions</td><td>No built-in support</td><td>Built-in support with middleware (e.g., redux-thunk)</td></tr>
</table></td></tr>
<tr><td>45. prop types in react(How to apply validation on props in react)</td><td>**PropTypes** is a simple, effective way to validate the types of props passed to your components.
It's useful for **catching bugs** early by ensuring the correct type of data is passed down to components.
For **small to medium-sized applications**, PropTypes is sufficient for prop validation.
For **larger applications**, consider using TypeScript for stricter, more powerful type checking across your codebase.</td></tr>
<tr><td>46. What are React Mixins ?</td><td>Mixins allow for code reuse across different components.
They can add functionality to components by providing shared behavior like lifecycle methods, event handling, etc.
In class components, mixins were used by including them in the mixins array or via React.createClass.</td></tr>
</table>

47. what are the different hooks you have used ?	**Common React Hooks Used**
	useState: Adds state to functional components.
	const [count, setCount] = useState(0);
	useEffect: Handles side effects like data fetching or subscriptions.
	useEffect(() => { fetchData() }, []);
	useContext: Accesses values from a React Context.
	const theme = useContext(ThemeContext);
	useReducer: Manages complex state with actions, like Redux.
	const [state, dispatch] = useReducer(reducer, initialState);
	useCallback: Memoizes functions to prevent unnecessary re-creations.
	const increment = useCallback(() => setCount(count + 1), [count]);
	useMemo: Memoizes computed values to optimize performance.
	const expensiveValue = useMemo(() => computeValue(), [count]);
	useRef: Stores mutable values that don't trigger re-renders, often used for DOM access.
	const inputRef = useRef();
	useLayoutEffect: Runs synchronously after DOM mutations for layout adjustments.
	useLayoutEffect(() => { const { width } = divRef.current.getBoundingClientRect(); }, []);
	useImperativeHandle: Customizes the instance value exposed to parent components using ref.
	useImperativeHandle(ref, () => ({ focus: () => inputRef.current.focus() }));

48. What are render props in react ?	A render prop is a function that returns JSX. It is passed as a prop to a component. The component using the render prop can invoke the function at any point in its render method, passing data or state to it.
49. What are the different types of exports and imports ?	**Named Exports:** Exporting multiple values (functions, variables, etc.) with names. export const add = ...; import { add } from './module'; **Default Exports:** Exporting a single value or function. export default greet; import greet from './module'; **Combining Exports:** A module can have both named and default exports. **Namespace Imports:** Importing all exports into a single object. import * as math from './math'; **Re-exporting:** Exporting from another module for aggregation. export { add } from './math';
50. What are the differences between create element vs clone element in react ?	**React.createElement** is used for creating new elements programmatically, often used internally by React or in custom renderers. **React.cloneElement** is used for cloning and modifying an existing React element, typically used for modifying props or children dynamically.

HTML & CSS

1. What is the difference between em and rem units in CSS?	Use em: When you want relative scaling within a specific context (e.g., for child elements relative to their parent). Use rem: For consistent, root-relative sizing, particularly for font sizes and layout dimensions.
2. Explain the purpose of the meta viewport tag in responsive design.	<meta viewport> tag is used in HTML to control the layout and scaling of a webpage on different devices, particularly on mobile devices.
3. How would you implement a responsive navigation bar using CSS?	A responsive navigation bar adapts its layout for different screen sizes, often switching between a horizontal menu for desktops and a dropdown or hamburger menu for smaller screens.
4. What are pseudo elements and pseudo-classes in CSS? Can you give examples?	**Feature** **Pseudo-Elements** **Pseudo-Classes** **Purpose** Element ke part ko target karta hai Element ki state ya interaction ko target karta hai **Syntax** ::pseudo-element :pseudo-class **Examples** ::before, ::after, ::first-letter :hover, :nth-child(), :focus
5. Explain the CSS Box Model and its components.	**content-box:** Default behavior; padding and border increase the total size. **border-box:** Ideal for fixed layouts, as padding and border are included in the specified size.
6. What is the difference between inline, inline-block, and block elements in HTML?	**Inline**: For small elements like links, icons, or text formatting. (span, a, img etc) **Inline-Block**: When inline alignment is needed with block-like styling. **Block**: For structural elements that occupy a full-width layout. (div, p etc)

7. What are media queries, and how would you use them in a project?	Media queries are a feature of CSS that allow you to apply styles based on the device's characteristics, such as screen size, resolution, orientation, or aspect ratio. They are primarily used to create responsive designs, ensuring a website looks good on various devices (desktop, tablet, mobile, etc.).
	
	format:
	@media (condition) {
	/* CSS styles */
	}

8. What is the difference between relative, absolute, fixed, and sticky positioning in CSS?	**relative**: Moves an element in relation to its original position. **absolute**: Removes the element from the flow and positions it relative to its closest positioned ancestor. **fixed**: Removes the element from the flow and keeps it fixed in the viewport, even during scrolling. **sticky**: Combines relative and fixed positioning, allowing an element to "stick" at a certain position after scrolling.
9. What is the Selectors in CSS?	**Most Used Selectors:** Class (.classname), ID (#idname), Child (>). **Combinators**: Descendant (space) vs Child (>). **Attribute Selectors**: Be clear about ^=, $=, *= meanings. **Practice Pseudo-Classes & Pseudo-Elements.**

JavaScript

1. What is the difference between var, let, and const in JavaScript?	Use let for variables that will change. Use const for values that should stay constant. Avoid using var unless necessary for legacy code.
2. Explain the concept of closures in JavaScript with an example.	Imagine you have a function inside another function. The inner function remembers the variables of the outer function, even after the outer function is done running. This "remembering" is called a closure.
3. How does the this keyword work in JavaScript?	The this keyword in JavaScript refers to the object that is currently executing the function. Its value depends on how and where the function is called, not where it is defined.
4. What are higher-order functions, and can you provide an example?	A higher-order function is a function that does at least one of the following: 1. Takes another function as an argument. 2. Returns a function as its result.
5. What is the difference between synchronous and asynchronous JavaScript?	**synchronous**: One task at a time (blocking). eg.: Loops, calculations. **asynchronous**: Multiple tasks can overlap (non-blocking). eg: API calls, timers, file reading.
6. Explain the difference between forEach and map methods in arrays.	**Feature** **forEach** **map** **Purpose** Perform an action on each element. Transform elements into a new array. **Return Value** undefined. A new array. **Chainable** No. Yes. **Original Array** May modify it (not recommended). Unchanged. **Use Case** Side effects (e.g., logging). Transforming data.
7. What is the purpose of promises, and how do you use them?	Promises are used to handle asynchronous operations in JavaScript, such as fetching data, reading files, or waiting for timers. **Lifecycle of a Promise** **Pending**: The initial state (neither fulfilled nor rejected). **Fulfilled**: The operation completed successfully (resolve called). **Rejected**: The operation failed (reject called). **How to Use Promises in JavaScript** **Create a Promise:** A Promise handles async tasks and has two outcomes: success (resolve) or failure (reject). **Consume a Promise:** Use .then() for success, .catch() for errors, and .finally() for cleanup. **Fetch Example**: Promises are great for APIs. **Promise.all**: Run multiple promises together.
8. What are arrow functions, and how do they differ from regular functions?	Arrow functions (=>) are a concise syntax for writing functions in JavaScript. They are typically used for shorter and simpler function expressions. **When use:** When you want a concise function syntax. When you need this to refer to the surrounding context. For simple functions like callbacks, array methods (map, filter).
9. How does event delegation work in JavaScript?	Event delegation is a technique where you add an event listener to a parent element instead of each child element. The event listener uses the bubbling phase to "catch" events triggered by child elements. **How Event Delegation Works** **Event Bubbling:** When an event occurs on a child element, it propagates upward through its ancestors until it reaches the document (parent → grandparent → ...). **Single Listener:** By attaching a listener to a parent element, you can handle events for all its child elements, even dynamically added ones.
10. What are REST APIs?	**REST APIs (Representational State Transfer APIs)** are a way for different software applications to communicate with each other over the internet. They follow a set of rules (REST principles) that make it easy to access, modify, or manipulate data. REST APIs provide a simple and flexible way to communicate between applications, commonly used in web development for accessing and managing data. **Key Features of REST APIs:-** 1. **Stateless**: Each request is independent; the server doesn't remember past interactions. 2. **Resource-Based**: Operates on resources like users, products, or posts, represented as URLs. 3. **HTTP Methods:** GET: Fetch data. POST: Create new data. PUT/PATCH: Update existing data. DELETE: Remove data. 4. **JSON Format**: Data is usually sent in JSON, a lightweight and readable data format. **Example of a REST API** **Endpoint (URL):** https://api.example.com/users **Requests:** GET: Fetch a list of users. POST: Add a new user. GET /users/1: Fetch a specific user (ID: 1). PUT /users/1: Update user with ID 1. DELETE /users/1: Remove user with ID 1.
11. How do you ensure the security of an API?	To ensure the security of an API, it's essential to use a combination of methods like **Use HTTPS (TLS/SSL)** **Authentication**, **Authorization**, **Encryption**, **Rate limiting**, **Use API Gateway**, **Input validation**, **Regular monitoring**. Always stay updated on security best practices to protect your API from various types of attacks.
12. Explain the purpose of HTTP status codes in API responses.	HTTP status codes are essential for effective communication between the client and server, ensuring smooth operation and easier debugging in API development. They help developers understand whether the request was successful, failed, or needs further action. These codes improve communication between the client and server, making it easier to handle errors and debug issues.
13. How would you optimize API performance?	To optimize API performance, focus on reducing response time, handling high traffic efficiently, and ensuring scalability through caching, efficient database queries, and load balancing. Optimizing an API ensures faster response times, better scalability, and an overall improved user experience. Here are the key strategies to achieve this: 1. Use Caching 2. Enable Compression 3. Minimize Payload Size 4. Implement Rate Limiting 5. Optimize Database Queries 6. Use Load Balancers 7. Use Asynchronous Processing 8. Optimize Authentication 9. Implement API Gateway 10. Use Content Delivery Networks (CDN) 11. Monitor and Log API Performance 12. Version Your APIs 13. Optimize Network Latency 14. Use GraphQL (Where Appropriate) 15. Minimize Middleware Overhead
14. What is the difference between GET and POST methods in REST APIs?	Use **GET** for retrieving data (read operations). GET is used when you want to fetch data from a server. Think of it as reading something, without making changes. Fetch data, like user details, products, etc. Use **POST** for sending data to create or update resources. Submit forms, create new records, upload files. POST is used when you want to send data to a server to create or update something. Think of it as writing something new.

React.js

1. What is the virtual DOM, and how does React use it?	
2. Explain the difference between functional components and class components in React.	
3. What is the purpose of useState and useEffect hooks?	
4. How does props differ from state in React?	
5. What are controlled and uncontrolled components in React?	
6. How do you handle forms in React?	
7. What is React context, and how does it help in state management?	
8. How would you optimize a React application for performance?	
9. What is the difference between useMemo and useCallback hooks?	
10. Explain React's lifecycle methods and their equivalents in hooks.	

Redux

1. What are the three core principles of Redux?	
2. Explain the structure of a Redux application.	
3. What is the role of actions in Redux? Can you provide an example?	
4. What is a reducer, and how does it work in Redux?	
5. How is the Redux store created, and how do you access its state?	
6. What is the purpose of connect() in Redux?	
7. What is the difference between mapStateToProps and mapDispatchToProps?	
8. How do you handle asynchronous operations in Redux?	
9. What is middleware in Redux, and can you give examples of common middleware?	
10. How would you use Redux Toolkit in a React application?	
11. Explain the purpose of selectors in Redux.	
12. What is the difference between Redux and React Context API?	
13. Explain how you would optimize a Redux-based application to prevent unnecessary renders.	